IUBAT-International University of Business Agriculture and Technology.

Final Assignment

Course Name: Programming in C++

Course Code: CSC 283

Semester: Summer Recess 2024

Submitted To

Mahmudul Hasan

Lecturer

Department of Computer Science and Engineering

Submitted By

Md. Abdullah Al Mukit

ID: 22103268

Program: BCSE

Section: A

Serial: 14

Date of Submission: 7/9/2024

1.Problem Definition

I will have to create a program which can use for a patient record management. The system should work in such a way where a receptionist can add/remove patient related information. We know that, patient have a lot of information like age, gender, disease, health issue, admit date, release date etc. By using C++ program, we will create it and their functionalities will work smoothly. This program is related to patient record management where a user can add, remove or edit their individual information. For this management we need some method and we are using our program to manipulate the information. So, from the code we can see the whole system or working functionalities.

CODE

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;
// Class representing a patient
class Patient {
public:
    string name;
    string patientID;
    string gender;
    string diagnosis;

    // Constructor
    Patient(string n, string id, string g, string d)
        : name(n), patientID(id), gender(g), diagnosis(d) {}

    // Display patient details
    void display() {
        cout << "Patient ID: " << patientID << "\n"
             << "Name: " << name << "\n"
             << "Gender: " << gender << "\n"
             << "Diagnosis: " << diagnosis << endl;
```

```
    }
};

// Utility function to validate patient ID format
bool validatePatientID(const string& id) {
    // Implement your own logic for ID validation (e.g., length, format)
    if (id.length() != 6 || !isdigit(id[0])) {
        return false;
    }
    return true;
}

// Class for managing patient records
class PatientManagementSystem {
private:
    vector<Patient> patients;

public:
    // Function to add a new patient
    void addPatient() {
        string name, patientID, gender, diagnosis;
        cout << "Enter Patient Name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter Patient ID (6 characters, first is digit): ";
        cin >> patientID;
        while (!validatePatientID(patientID)) {
            cout << "Invalid Patient ID format! Try again: ";
            cin >> patientID;
        }
        cout << "Enter Gender: ";
        cin >> gender;
        cout << "Enter Diagnosis: ";
        cin.ignore();
```

```
    getline(cin, diagnosis);

    // Check for unique Patient ID
    for (auto& patient : patients) {
        if (patient.patientID == patientID) {
            cout << "Patient ID already exists! Cannot add duplicate records." << endl;
            return;
        }
    }

    patients.push_back(Patient(name, patientID, gender, diagnosis));
    cout << "Patient record added successfully.\n";
}

// Function to update an existing patient's record
void updatePatient() {
    string patientID;
    cout << "Enter Patient ID to update: ";
    cin >> patientID;

    for (auto& patient : patients) {
        if (patient.patientID == patientID) {
            cout << "Enter New Name: ";
            cin.ignore();
            getline(cin, patient.name);
            cout << "Enter New Gender: ";
            cin >> patient.gender;
            cout << "Enter New Diagnosis: ";
            cin.ignore();
            getline(cin, patient.diagnosis);
            cout << "Patient record updated successfully.\n";
            return;
        }
    }
```

```
    cout << "Patient record not found.\n";
}

// Function to delete a patient record
void deletePatient() {
    string patientID;
    cout << "Enter Patient ID to delete: ";
    cin >> patientID;

    auto it = remove_if(patients.begin(), patients.end(), [&](Patient& p) {
        return p.patientID == patientID;
    });

    if (it != patients.end()) {
        patients.erase(it, patients.end());
        cout << "Patient record deleted successfully.\n";
    } else {
        cout << "Patient record not found.\n";
    }
}

// Function to search for a patient by ID
void searchPatient() {
    string patientID;
    cout << "Enter Patient ID to search: ";
    cin >> patientID;

    for (auto& patient : patients) {
        if (patient.patientID == patientID) {
            patient.display();
            return;
        }
    }
```

```
        cout << "Patient record not found.\n";
    }

    // Function to generate report of all patients
    void generateReport() {
        if (patients.empty()) {
            cout << "No patients in the system.\n";
            return;
        }

        cout << "\nPatient Records:\n";
        for (auto& patient : patients) {
            patient.display();
            cout << "-------------------\n";
        }
    }
};

int main() {
    PatientManagementSystem system;
    int choice;

    while (true) {
        cout << "\nPatient Record Management System\n";
        cout << "1. Add Patient\n";
        cout << "2. Update Patient\n";
        cout << "3. Delete Patient\n";
        cout << "4. Search Patient\n";
        cout << "5. Generate Report\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
```

```
        switch (choice) {
            case 1:
                system.addPatient();
                break;
            case 2:
                system.updatePatient();
                break;
            case 3:
                system.deletePatient();
                break;
            case 4:
                system.searchPatient();
                break;
            case 5:
                system.generateReport();
                break;
            case 6:
                cout << "Exiting system...\n";
                return 0;
            default:
                cout << "Invalid choice! Try again.\n";
        }
    }

    return 0;
}
```

OUTPUT:

```
Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: male
Enter Diagnosis: blood test
Patient record added successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 2
Enter Patient ID to update: 1Mukit
Enter New Name: 2Harun
Enter New Gender: male
Enter New Diagnosis: blood cancer
Patient record updated successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 3
Enter Patient ID to delete: 1Mukit
Patient record deleted successfully.
```

```
6. Exit
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: male
Enter Diagnosis: blood test
Patient record added successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 4
Enter Patient ID to search: 1Mukit
Patient ID: 1Mukit
Name: Mukit
Gender: male
Diagnosis: blood test
```

```
Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 5

Patient Records:
Patient ID: 1Mukit
Name: Mukit
Gender: male
Diagnosis: blood test
------------------

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 6
Exiting system...

Process returned 0 (0x0)   execution time : 48.017 s
Press any key to continue.
```

2.System Requirements

From this page we will choose the any System:

```
"D:\All Programme\Unt.exe"

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice:
```

1.For adding patient, we will get:

```
"D:\All Programme\Unt.exe"

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: male
Enter Diagnosis: Blood Test
Patient record added successfully.
```

2.For updating patient, we will get:

```
6. Exit
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: male
Enter Diagnosis: Blood test
Patient record added successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 2
Enter Patient ID to update: 1Mukit
Enter New Name: 2Harun
Enter New Gender: male
Enter New Diagnosis: cancer
Patient record updated successfully.
```

3.For deleting patient, we will get:

```
Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 3
Enter Patient ID to delete: 1Mukit
Patient record deleted successfully.
```

4.For searching patient, we will get:

```
6. Exit
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: Male
Enter Diagnosis: Blood test
Patient record added successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 4
Enter Patient ID to search: 1Mukit
Patient ID: 1Mukit
Name: Mukit
Gender: Male
Diagnosis: Blood test
```

5.For generating patient report, we will get:

```
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: male
Enter Diagnosis: blood test
Patient record added successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 5

Patient Records:
Patient ID: 1Mukit
Name: Mukit
Gender: male
Diagnosis: blood test
--------------------
```

6.For exiting, we will get:

```
Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 6
Exiting system...

Process returned 0 (0x0)   execution time : 48.017 s
Press any key to continue.
```

3.Implementaion Steps

1.For designing the program structure, including classes like patient and function/method like addPatient,deletePatient. Here below, we will see the whole method-

```
// Class representing a patient
class Patient {
public:
    string name;
    string patientID;
    string gender;
    string diagnosis;

    // Constructor
    Patient(string n, string id, string g, string d)
        : name(n), patientID(id), gender(g), diagnosis(d) {}

    // Display patient details
    void display() {
        cout << "Patient ID: " << patientID << "\n"
             << "Name: " << name << "\n"
             << "Gender: " << gender << "\n"
             << "Diagnosis: " << diagnosis << endl;
    }
};

// Utility function to validate patient ID format
bool validatePatientID(const string& id) {
    // Implement your own logic for ID validation (e.g., length, format)
    if (id.length() != 6 || !isdigit(id[0])) {
        return false;
    }
    return true;
}
```

```
// Class for managing patient records
class PatientManagementSystem {
private:
    vector<Patient> patients;

public:
    // Function to add a new patient
    void addPatient() {
        string name, patientID, gender, diagnosis;
        cout << "Enter Patient Name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter Patient ID (6 characters, first is digit): ";
        cin >> patientID;
        while (!validatePatientID(patientID)) {
            cout << "Invalid Patient ID format! Try again: ";
            cin >> patientID;
        }
        cout << "Enter Gender: ";
        cin >> gender;
        cout << "Enter Diagnosis: ";
        cin.ignore();
        getline(cin, diagnosis);
```

2.For implementing the data management functionalities for adding, updating and deleting patient records, we are using this type of code in C++. Here below:

```
int main() {
    PatientManagementSystem system;
    int choice;

    while (true) {
        cout << "\nPatient Record Management System\n";
        cout << "1. Add Patient\n";
        cout << "2. Update Patient\n";
```

```
        cout << "3. Delete Patient\n";
        cout << "4. Search Patient\n";
        cout << "5. Generate Report\n";
        cout << "6. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                system.addPatient();
                break;
            case 2:
                system.updatePatient();
                break;
            case 3:
                system.deletePatient();
                break;
            case 4:
                system.searchPatient();
                break;
            case 5:
                system.generateReport();
                break;
            case 6:
                cout << "Exiting system...\n";
                return 0;
            default:
                cout << "Invalid choice! Try again.\n";
        }
    }

    return 0;
}
```

3.For searching algorithms to retrieve patient records, we can adjust it with a specific criterion. We need some algorithm like pop,s take, push etc. This type of algorithm as well as also bubble short, linear search.

4.Creating a report generation functions to display information by using this code and will show output of the code:

```
    // Function to generate report of all patients
    void generateReport() {
        if (patients.empty()) {
            cout << "No patients in the system.\n";
            return;
        }

        cout << "\nPatient Records:\n";
        for (auto& patient : patients) {
            patient.display();
            cout << "-------------------\n";
        }
    }
};
```

Output:

```
Enter your choice: 1
Enter Patient Name: Mukit
Enter Patient ID (6 characters, first is digit): 1Mukit
Enter Gender: male
Enter Diagnosis: blood test
Patient record added successfully.

Patient Record Management System
1. Add Patient
2. Update Patient
3. Delete Patient
4. Search Patient
5. Generate Report
6. Exit
Enter your choice: 5

Patient Records:
Patient ID: 1Mukit
Name: Mukit
Gender: male
Diagnosis: blood test
-------------------
```

5.By testing the system thoroughly and we find that all functionalities work as expected and handling the edge cares smoothly and properly.

Conclusion: This patient record management is very helpful for the hospital community. They can admit the patient or record the patient information. As a result, a user will feel comfortable while collecting the patient information. So, as we expect that this patient record management will give a monotonous perspective for a program any type of situation.